

实验三：类和对象（二）

一、实验目的

1. 掌握构造函数、拷贝构造函数、析构函数的定义与使用。
2. 掌握类、成员函数、对象的定义方法。
3. 学习使用静态成员、内联成员函数，掌握普通拷贝构造函数的使用。
4. 掌握对象成员的使用方法。
5. 掌握多文件结构（.h 声明 + .cpp 实现 + main.cpp）程序设计方法。
6. 进一步熟悉 Visual C++ 的编译连接过程，掌握 Visual C++ 的调试方法

二、实验要求

1. 创建一个学生链表，进行链表的插入、删除、查找操作，要求：
 - (1) 使用函数模板；
 - (2) 使用 new 和 delete 进行动态内存空间的分配和释放。
2. 使用类创建一个学生链表，进行链表的插入、删除、查找操作。

三、实验原理

1. 指针与引用的原理

指针（Pointer）

- (1) 指针是一个独立的变量，存储的是另一个变量的内存地址
- (2) 指针可以改变指向，可以为空（nullptr）
- (3) 访问目标数据需要解引用操作（*运算符）

引用（Reference）

- (1) 引用是变量的别名，与目标变量共享同一内存空间
- (2) 引用必须在定义时初始化，且不能更改绑定
- (3) 使用引用时语法与普通变量相同，更加直观

2. 函数重载与函数模板原理

函数重载（Function Overloading）

- (1) 同一作用域内，函数名相同但参数列表不同（参数类型、个数、顺序）
- (2) 编译器根据实参类型自动匹配最合适的函数版本
- (3) 属于静态多态（编译期绑定）

函数模板（Function Template）

- (1) 使用泛型参数（`typename T`）定义通用函数框架
- (2) 编译器根据调用时的实参类型自动推导生成具体函数
- (3) 代码复用性高，减少重复代码

3.C++动态内存管理原理

new/delete 机制

- (1) **new**: 分配内存 + 调用构造函数初始化
- (2) **delete**: 调用析构函数清理 + 释放内存

4.链表数据结构原理

节点结构

数据域	指针域
(data)	(next)
Student	Node*

基本操作时间复杂度

- (1) 头部插入: $O(1)$
- (2) 尾部插入: $O(n)$
- (3) 按值查找: $O(n)$
- (4) 按值删除: $O(n)$

四、实验内容与步骤

实验内容一：

创建一个学生链表，进行链表的插入、删除、查找操作，要求：

- (1) 使用函数模板；
- (2) 使用 `new` 和 `delete` 进行动态内存空间的分配和释放。

【实验步骤】

Step 1: 创建项目

创建名为 `StudentList` 的 C++ 项目。

开始使用



克隆存储库(C)

从 GitHub 或 Azure DevOps 等联机存储库获取代码



打开项目或解决方案(P)

打开本地 Visual Studio 项目或 .sln 文件



打开本地文件夹(F)

导航和编辑任何文件夹中的代码



创建新项目(N)

选择具有代码基架的项目模板以开始



搜索模板(Alt+S)(S) [全部清除\(C\)](#)

C++ 所有平台(P) 控制台

空项目
使用适用于 Windows 的 C++ 从头开始操作。不提供基础文件。
C++ Windows 控制台

控制台应用
在 Windows 终端运行代码。默认打印 "Hello World"。
C++ Windows 控制台 **1**

CMake 项目
生成不依赖于 .sln 或 .vcxproj 文件的新式跨平台 C++ 应用。
C++ Windows Linux 控制台

Windows 桌面向导
使用向导自行创建 Windows 应用。
C++ Windows 桌面 控制台 库

共享项目
使用“共享条目”项目在多个项目之间共享文件。
C++ Windows Android iOS Linux 桌面 控制台 库
UWP 游戏 移动

生成文件项目

2

[上一步\(B\)](#) [下一步\(N\)](#)

配置新项目

控制台应用 C++ Windows 控制台

项目名称(N) **1**

位置(L)

解决方案名称(M)

☒ 将解决方案和项目放在同一目录中(D)

项目 将在 "C:\Users\15007\source\repos\StudentList\" 中创建

[上一步\(B\)](#) [创建\(C\)](#) **2**



Step2: 代码编写

```
#include <iostream>
#include <string>
using namespace std;
```

```
// 学生结构体
struct Student
{
    int id;
    string name;
    Student* next;
};
```

```
// 1. 普通函数: 创建节点
Student* createNode(int id, string name)
{
    【1】 // 用 new 创建节点
    node->id = id;
    node->name = name;
    node->next = nullptr;
    return node;
}
```

```
// 2. 函数模板: 插入节点
template <typename T>
void insert(T*& head, int id, string name)
{
    T* newNode = createNode(id, name);
    if ( 【2】 ) // 判断链表是否为空
    {
        head = newNode;
        return;
    }
}
```

```

    }
    T* p = head;
    while (p->next != nullptr)
        p = p->next;
    【3】; // 把新节点链到尾部
}

```

// 3. 函数重载：按学号删除

```

bool remove(Student*& head, int id)
{
    if (head == nullptr) return false;

    Student* p = head;
    Student* pre = nullptr;

    if (p->id == id)
    {
        head = head->next;
        【4】; // 删除头节点
        return true;
    }

    while (p != nullptr && p->id != id)
    {
        pre = p;
        p = p->next;
    }
    if (p == nullptr) return false;

    pre->next = p->next;
    delete p;
    return true;
}

```

// 4. 函数重载：按姓名删除

```

bool remove(Student*& head, string name)
{
    // 请自行补全（仿照按学号删除）
    【5】
}

```

// 5. 函数模板：按学号查找

```

template <typename T>
T* find(T* head, int id)

```

```

{
    T* p = head;
    while (p != nullptr)
    {
        if (【6】) // 判断 id 是否相等
            return p;
        p = p->next;
    }
    return nullptr;
}

```

// 6. 普通函数：遍历显示链表

```

void show(Student* head)
{
    Student* p = head;
    while (p != nullptr)
    {
        cout << "学号: " << p->id << " 姓名: " << p->name << endl;
        【7】; // p 指向下一个节点
    }
}

```

// 释放链表

```

void freeList(Student*& head)
{
    Student* p = head;
    while (p != nullptr)
    {
        Student* q = p->next;
        delete p;
        p = q;
    }
    head = nullptr;
}

```

// ===== 主函数（必须调用所有功能） =====

```

int main()
{
    Student* head = nullptr;

    // 调用 createNode
    Student* tempNode = createNode(1000, "临时节点");
    cout << "临时节点: " << tempNode->id << " " << tempNode->name << endl;
    delete tempNode;
}

```

```

// 调用 insert
insert(head, 1001, "张三");
insert(head, 1002, "李四");
insert(head, 1003, "王五");
insert(head, 1004, "赵六");
cout << "\n===== 插入后 =====" << endl;
show(head);

// 调用 remove(学号)
cout << "\n===== 按学号删除 1002 =====" << endl;
remove(head, 1002);
show(head);

// 调用 remove(姓名)
cout << "\n===== 按姓名删除赵六 =====" << endl;
remove(head, "赵六");
show(head);

// 调用 find
cout << "\n===== 查找学号 1003 =====" << endl;
Student* res = find(head, 1003);
if (res)
    cout << "查找成功: " << res->id << " " << res->name << endl;

// 调用 show
cout << "\n===== 最终链表 =====" << endl;
show(head);

freeList(head);
return 0;
}

```

Step3: 运行结果

```

===== 调用createNode创建临时节点 =====
临时节点: 1000 临时节点

===== 调用insert插入后 =====
学号: 1001 姓名: 张三
学号: 1002 姓名: 李四
学号: 1003 姓名: 王五
学号: 1004 姓名: 赵六

===== 调用重载remove(学号1002) =====
学号: 1001 姓名: 张三
学号: 1003 姓名: 王五
学号: 1004 姓名: 赵六

===== 调用重载remove(姓名赵六) =====
学号: 1001 姓名: 张三
学号: 1003 姓名: 王五

===== 调用find查找学号1003 =====
查找成功: 1003 王五

===== 最终调用show显示全部 =====
学号: 1001 姓名: 张三
学号: 1003 姓名: 王五

===== 调用freeList释放链表完成 =====

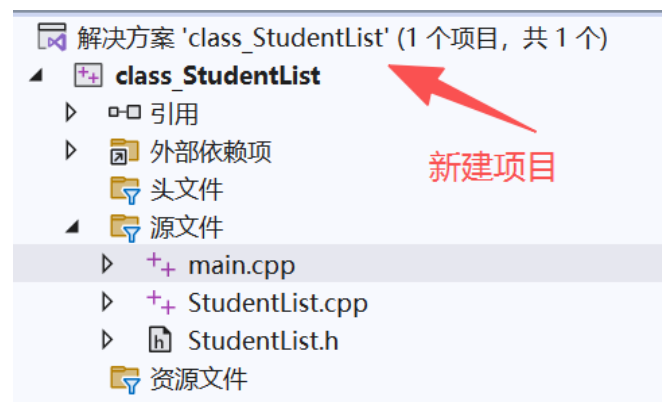
```

实验内容二:

使用类创建一个学生链表，进行链表的插入、删除、查找操作。

【实验步骤】

Step1: 创建项目



Step2: 定义学生结点类，链表类

创建 StudentList.h 文件

```
#pragma once
```

```
#include <iostream>
```

```

#include <string>
using namespace std;

// 学生节点类
class Student {
public:
    int id;
    string name;
    Student* next;

    // 构造函数声明
    【填空 1】(int id_, string name_);
};

// 链表管理类
class StudentList {
private:
    【填空 2】; // 链表头指针，类型为 Student*

public:
    // 构造函数
    【填空 3】();

    // 成员函数声明
    void insert(int id, string name);
    bool removeById(int id);
    Student* findById(int id);
    void showAll();

    // 析构函数
    【填空 4】();
};

```

Step3: 补全【填空】类定义中的函数体

创建 StudentList.cpp 文件（类实现）

```

#include "StudentList.h"

// Student 构造函数实现
Student::Student(int id_, string name_) {
    id = id_;
    name = name_;
    【填空 5】; // next 置空
}

```

```

// 链表构造函数
StudentList::StudentList() {
    【填空 6】; // head 初始化为空
}

// 尾插法插入学生
void StudentList::insert(int id, string name) {
    Student* newNode = 【填空 7】; // new 创建节点
    if (head == nullptr) {
        head = newNode;
        return;
    }
    Student* p = head;
    while (【填空 8】) { // 找到尾节点
        p = p->next;
    }
    【填空 9】; // 新节点链到尾部
}

// 按学号删除
bool StudentList::removeById(int id) {
    if (head == nullptr) return false;

    Student* p = head;
    Student* pre = nullptr;

    if (p->id == id) {
        head = head->next;
        【填空 10】; // 删除节点
        return true;
    }

    while (p != nullptr && p->id != id) {
        pre = p;
        p = p->next;
    }
    if (p == nullptr) return false;

    pre->next = p->next;
    delete p;
    return true;
}

// 按学号查找

```

```

Student* StudentList::findById(int id) {
    Student* p = head;
    while (p != nullptr) {
        if (【填空 11】){// 匹配学号
            return p;
        }
        p = p->next;
    }
    return 【填空 12】; // 没找到返回空
}

// 遍历显示
void StudentList::showAll() {
    Student* p = head;
    while (p != nullptr) {
        cout << "学号: " << p->id << " 姓名: " << p->name << endl;
        【填空 13】; // 指针后移
    }
}

// 析构函数：释放链表
StudentList::~StudentList() {
    Student* p = head;
    while (p != nullptr) {
        Student* q = p->next;
        【填空 14】;
        p = q;
    }
    head = nullptr;
}

```

Step4: 调用所有功能（进行链表的插入、删除、查找操作）

创建 main.cpp 文件（测试调用）

```

#include "StudentList.h"
using namespace std;

```

```

int main() {
    // 创建链表对象
    【填空 15】 list;

    // 插入学生
    list.insert(1001, "张三");
    list.insert(1002, "李四");
}

```

```

list.insert(1003, "王五");

cout << "==== 插入后 =====> << endl;
    【填空 16】; // 调用 showAll 显示

// 删除学号 1002
cout << "\n==== 删除后 =====> << endl;
list.removeById(1002);
list.showAll();

// 查找 1003
cout << "\n==== 查找结果 =====> << endl;
Student* res = 【填空 17】;
if (res) {
    cout << "找到: " << res->name << endl;
} else {
    cout << "未找到" << endl;
}

return 0;
}

```

Step5: 编译运行

```

==== 插入后链表 ====
学号: 1001      姓名: 张三
学号: 1002      姓名: 李四
学号: 1003      姓名: 王五
学号: 1004      姓名: 赵六

==== 删除学号 1002 ====
学号: 1001      姓名: 张三
学号: 1003      姓名: 王五
学号: 1004      姓名: 赵六

==== 查找学号 1003 ====
找到学生: 王五

```

实验提交要求

完成所有代码填空，正确编译、运行并截图结果。

记录实验中遇到的错误及解决方法。

提交源代码、运行截图、实验报告

答案页

第一题：

- 【1】 Student* node = new Student;
- 【2】 head == nullptr
- 【3】 p->next = newNode
- 【4】 delete p
- 【5】 完整实现按姓名删除（参照按学号版本）
- 【6】 p->id == id
- 【7】 p = p->next

第二题：

- 【1】 Student
- 【2】 Student* head
- 【3】 StudentList
- 【4】 ~StudentList
- 【5】 next = nullptr
- 【6】 head = nullptr
- 【7】 new Student(id, name)
- 【8】 p->next != nullptr
- 【9】 p->next = newNode
- 【10】 delete p
- 【11】 p->id == id
- 【12】 nullptr
- 【13】 p = p->next
- 【14】 delete p
- 【15】 StudentList
- 【16】 list.showAll()
- 【17】 list.findById(1003)